

EdgePlex: Decomposing the Provider Edge for Flexibilty and Reliability

Angela Chiu[†], Vijay Gopalakrishnan[†], Bo Han[†], Murad Kablan[‡], Oliver Spatscheck[†]
Chengwei Wang[†], Yang Xu[†]
[†]AT&T Labs - Research, [‡]University of Colorado

ABSTRACT

The service provider edge is responsible for connecting customers using standard protocols such as IP and BGP to the service providers internal network while enforcing service specific policies and service guarantees. Today this function is performed by the Provider Edge Router (PE). The specialized nature of the PE, however, restricts operational flexibility and their monolithic design impacts reliability. In this paper, we propose a new edge router architecture following SDN and NFV principals called EdgePlex. EdgePlex is a distributed system where the functions performed by a traditional edge router are decomposed and assigned to different elements in the system. A key aspect of our design is the use of a sandboxed environment (through the use of virtual machines) per customer. This gives EdgePlex the ability to isolate customers from one-another and independently move customers within and across EdgePlex platforms. We describe the architecture and a prototype implementation of EdgePlex. We perform detailed experiments using this prototype and show that EdgePlex is able to saturate the server in terms of throughput while having acceptable latency and jitter overheads. Our results lead us to believe that the EdgePlex design not only addresses the limitations of existing routers, but is also viable and can meet performance demands of production networks.

Categories and Subject Descriptors

C.2.6 [**Internetworking**]: Routers; C.2.3 [**Network Operations**]: Network Management; C.4 [**Performance of Systems**]: Reliability, availability, and serviceability

General Terms

Design, Reliability, Performance

Keywords

SDN, Virtualization, Provider Edge Router

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
SOSR2015, June 17 – 18, 2015, Santa Clara, CA, USA
Copyright 2015 ACM. ISBN 978-1-4503-3451-8/15/06...$15.00
DOI: http://dx.doi.org/10.1145/2774993.2775010.

1. INTRODUCTION

IP networks have traditionally been designed and architected such that the core of the network performs simple tasks quickly, while pushing the more complex tasks to the edge. This allows for the core to perform fast packet forwarding (e.g., using Multi-Protocol Label Switching (MPLS)) while the edge aggregates traffic, exchanges control plane information, performs routing, and supports other features demanded by the services (access control, Quality of Service (QoS), etc). To enable this, service providers use specialized, physical devices known as Provider Edge (or PE) routers that allow customers to connect to the service provider's network. A single PE router aggregates traffic from hundreds of customers and funnels the traffic into the core network.

The specialized and physical nature of PEs, however, introduces challenges in how service providers manage and operate them. Since many customers share a router, they share the same feature sets and are all impacted when changes are applied to a router. There is also generally a static mapping between a PE and a customer. This is because customers can not be moved without prolonged service interruptions or without affecting other customers configured on that router. As a result, customers cannot quickly recover during a network failure. Resources on the PE router cannot be optimized dynamically. For example, today's PE routers can exhaust their capacity in terms of bandwidth, routing state or physical port count. The static nature of PE assignments forces service providers to ensure that none of those resources are exhausted during the entire multi-year life cycle of a device.

In this paper, we examine how Network Function Virtualization (NFV) and Software Defined Networking (SDN) can be applied to edge routers to alleviate these challenges. To that end, we propose EdgePlex, a novel distributed architecture that uses servers, switches, and advances in virtualization and high speed software packet forwarding to realize PE router functionality. The switches interconnect the servers and connect them to the customers and the core network. The servers host virtual machines (VM), each of which represents a single customer endpoint. Associating each customer with a VM gives independent control over each customer, the ability to sandbox a customer from other customers, and to migrate customers. In addition, we can leverage a rich set of existing VM tools to manage customers.

There are, however, challenges in terms of scale, performance, and fault tolerance. We address these by intelligently partitioning functions across elements in EdgePlex. For example, we split control plane processing between a sin-

gle central entity and multiple customer facing VMs. This allows scaling from the core network perspective while allowing for customers to be migrated with minimal impacts. Similarly, we distribute packet forwarding across the customer VMs to take advantage of parallel processing and increase performance. While not a requirement, our design also allows for offloading packet forwarding to switches when capable and possible.

There are several key design aspects of EdgePlex that set it apart from alternate proposals. Our use of virtualization and commodity servers dramatically improves the flexibility and manageability of PE routers. Service providers can quickly and cost-effectively provision customers when and where needed. The isolation of customers from each other opens new options for service providers; from providing customized services to each customer, to alternate operational models. For example, it is now possible for service providers to provide on-demand connectivity to customers, run different versions of protocols for different customers without impacting each other, roll out updates at a finer granularity, or perform maintenance at a specific customer's convenience rather than at fixed times. Having the flexibility to run a different protocol version for each customer will be particularly important as service provider networks transition towards a fully software-defined environment. This is because it decouples the speed of SDN adoption by individual customers and within the provider's network, and thus leads to an incremental path for SDN deployment in service providers' networks.

In this paper we describe the architecture, design, and prototype implementation, as well as initial experimental results to demonstrate the efficacy of EdgePlex design. We use a combination of open source and custom-built software in our implementation. Our results demonstrate that we are able to get close to 10Gbps throughput from a single server just using software forwarding. We also show that we are able to leverage existing VM tools to provide fault tolerance. Our implementation and results lead us to conclude that the EdgePlex architecture is not only feasible, but is also scalable. It provides the desired performance characteristics while immensely improving flexibility and manageability.

2. RELATED WORK

In this section, we briefly review the existing work related to EdgePlex, which can be classified into three categories: virtualization, software packet processing, and customer migration on traditional routers.

Virtualization is the enabling technology behind the success of cloud computing and network function virtualization. In popular solutions, such as KVM [18], Xen [9] and VMWare [7], a hypervisor runs on servers and allows for the creation of multiple VMs, each running a separate instance of operating system. Multiple VMs can share the network devices on the physical host with I/O sharing technologies such as SR-IOV [5] and VMDq [6]. To bring virtualization to the network, NetVM [15] builds mechanisms across a KVM hypervisor and VMs to reduce data copies and locking overheads in inter-VM communications. ClickOS [20] provides a platform for building small middlebox VMs using Click in Xen hypervisor. Both Cisco and Juniper have virtualized instances of their physical routers [10, 16] using many of these recent capabilities. The key components of EdgePlex also rely on many of the advances in virtualization technology for their efficient functioning.

Previous work and library tools for software packet processing form the basis of many virtualized network elements. Click [19] and RouteBricks [12] are among the early research endeavors to enable software packet processing. PacketShader [14] exploits the parallel processing capability of GPU to accelerate packet processing in software routers. Recently, technologies such as Intel's DPDK [1], PFQ [3], PF_RING [4], and Netmap [22] have emerged. These provide programming libraries to build fast network I/O applications. Open vSwitch [2], Hyper-Switch [21] and VALE [23] build software switches in a hypervisor to create an Ethernet network among VMs. EdgePlex leverages fast software packet processing to achieve its design goals.

The need to dynamically move customers from one router to another has been recognized even before the use of virtual machines became widespread. Sebos et al. [24] proposed the idea of migrating customers affected by access router failures and demonstrated the feasibility using Linux-based software routers. Agarwal et al. [8] built on the idea in [24] to propose the concept of a router farm which, similar to a server farm, can be used to reduce customer outages during router maintenance. VROOM [25] allows "hot" migration of customers from one device to another and uses the idea of programmable transport networks as proposed in [24] to move customer connections. Keller et al. [17] proposed the idea of router grafting, where parts of a router are removed from one router and merged into another. In contrast to these approaches, we propose to use the live migration ability of virtual machines to move customers, not only between logical PEs but also across servers within the same logical PE. Moreover, EdgePlex provides separation between and control of customers at a much finer granularity than any of these approaches.

3. THE EDGEPLEX ARCHITECTURE

We discuss the architecture of EdgePlex in this section. Drawing inspiration from the success of cloud computing platforms, the goal of our approach is to leverage advances in servers, switches and virtualization technology to replace appliance-based edge routers. To that end, our design of EdgePlex architecture is presented in Figure 1.

As shown in the figure, EdgePlex is a distributed platform consisting of commodity servers and switches. The switches interconnect the servers with each other and provide connectivity to the customers and to the core network. The servers support virtual machines by running hypervisors and hosting one or more virtual machines. The key novelty in the EdgePlex architecture is that each customer is assigned a virtual machine — which we call a PortVM (or PVM) — to which they connect. While not a one-to-one map, a PVM can be likened to the physical port on traditional hardware routers. The PVM stores the customer specific configuration like routing, access control, and quality-of-service information. It also stores other control- and data-plane state for the customer like its routing and forwarding tables. Thus, if a customer uses BGP (as is usually the case) for control plane, the TCP session associated with their BGP process terminates in the PVM. Associating each customer with a VM and storing their state in that VM gives us independent control over each customer. It provides operators with the

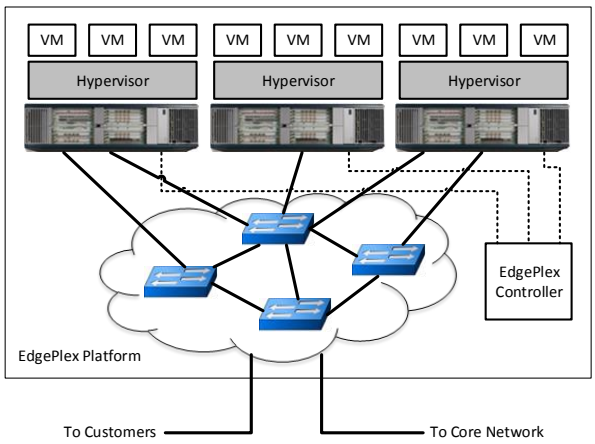


Figure 1: Architecture of EdgePlex platform.

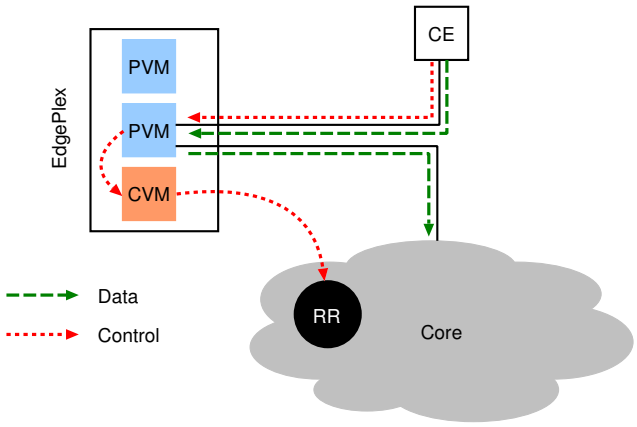


Figure 2: Split of control- and data-plane functions in EdgePlex.

new ability to sandbox and migrate individual customers with limited or no impact to other customers.

While this handles communications on the customer side, we still need to deal with communications with the core of the network. The simplest solution would be to allow each PVM to communicate both control- and data-plane exchanges with the rest of the network. In essence, each PVM would behave like a virtualized PE router with just one customer configured. This, however, has control plane implications and scalability limitations. Since each PVM has to have at least one BGP session back into the network (assuming the existence of BGP route reflectors), we now start scaling the number of BGP sessions from number of PE routers (order of hundreds to a few thousands) to number of customer PVMs (which could be tens to hundreds of thousands). In addition to BGP sessions, it has implications for MPLS label distribution because we need a different MPLS label per PVM and that information has to be distributed to every other PVM.

To address this, we introduce another VM called a ControlVM (or CVM). The controlVM (much like the loopback address in traditional routers) "represents" the PE from the control plane perspective on the core network. Thus it maintains BGP sessions with the route reflector and converses other control plane protocols (interior gateway protocols like OSPF, label distribution protocol for MPLS, etc.) with the core. The CVM then relays the necessary control plane information to and from each PVM. We assume that there is a single CVM per EdgePlex instance. Since the failure of a CVM can cause the core to renegotiate labels and redistribute routes, we use an active backup for the CVM. This can be easily achieved using existing VM redundancy techniques (e.g., Micro-Checkpointing in KVM). Our results in Section 4.4 show that this approach is also effective. Thus the use of CVM, while simple, is sufficient to overcome the possible control plane scaling limitations. Moreover, the use of CVM does not restrict or hinder flexibility in managing customer PVMs within an instance of EdgePlex.

Figure 2 shows the overall split in control- and data-plane functions in EdgePlex. We make the simplifying assumption that customers use Ethernet to connect to the service provider's network and that each customers traffic is separated using VLANs. This simplifies routing customer traffic to the corresponding PVM because the switches and hypervisors in the EdgePlex platform can be configured to terminate each customer's VLAN in their PVM. As discussed previously, and as shown in Figure 2, customers' control plane state terminates at their PVM. The PVM then exchanges relevant information to and from the CVM. The CVM has control plane sessions representing the EdgePlex PE with the core network. The exchange of control plane information between the PVM and CVM can be achieved through iBGP or through other custom protocols.

The PVM, by default as shown in Figure 2, is also responsible for the data forwarding for each customer. The PVM identifies the appropriate forwarding rule, makes the required header modifications (e.g., applying the right MPLS labels), and forwards the packets. While not essential, the EdgePlex architecture allows for alternate forwarding strategies such as offloading the forwarding of packets to the hypervisor or even to the physical switch to take advantage of faster packet forwarding capabilities.

Finally, as shown in Figure 1, given that EdgePlex is a distributed platform, we make use of a controller that manages this platform. The main role of the controller is to configure customers by creating their PVMs and configuring connectivity in the hypervisor and in the switches. It is also responsible for monitoring the functioning of the platform and take action for example, react to failures, load balance the platform, etc.

3.1 Strengths of EdgePlex Architecture

Having described the EdgePlex architecture, we discuss the strengths of the proposed design. Our proposed design addresses many of the shortcomings of existing edge routers, especially in terms of everyday operation and management, customer isolation, supporting new services, and providing a cost-efficient path for gradually introducing alternate networking technologies.

Intelligent Task Partitioning: We intelligently partition router functions across the elements in EdgePlex. We split control plane processing between the central CVM and all the PVMs. This allows scaling from the core network perspective while allowing for customers to be migrated with minimal impacts. Similarly, we distribute packet forwarding across the PVMs to take advantage of parallel processing

and increase performance. We also allocate tasks based on the entity's strengths. For example, control plane processing (e.g., route computation, BGP exchanges) is computationally heavy. Hence, we retain control plane processing on the servers and VMs. Physical switches support high-speed packet switching and forwarding. Thus when capable and possible, switches can be used to offload packet forwarding.

Efficient Fault Tolerance: Since EdgePlex is made up of commodity elements, failure of individual elements is going to be the norm. The design of EdgePlex allows us to minimize the impacts of faults. As discussed previously, we use existing redundancy techniques for CVM. We do not use them for PVMs because of the number of PVMs, and the limited impact of its failure. Only the customer configured on the PVM will lose connectivity when a PVM fails. However, we can quickly create a new VM in its place and restore connectivity. Similarly if a server fails, only customers on that server need to be handled. The impact of a switch failing depends on how the switches and servers are interconnected; however, redundant connections between servers and switches can help handle switch failures. Most importantly, we can handle failures (or maintenance) locally without having to move customers between remote PEs.

Easier Capacity Planning: Our use of virtualization, and our design in particular, dramatically simplifies capacity planning for PE routers. Since connecting a customer really requires just spinning up a VM for the customer, it allows service providers to quickly and cost-effectively provision customers when and where needed. Providers do not have to carefully plan the specific device that a customer has to connect. The elastic scaling that comes from the use of commodity servers and virtual machines allows service providers to elastically allocate capacity, making it capitally efficient. Thus when extra capacity is needed, more servers can be included into the platform; when capacity is no longer needed, these servers can be used for other applications.

Flexible Management: EdgePlex allows operators significantly more flexibility in managing customer connectivity. Unlike today, they can roll out updates in a finer granularity, by starting off with a few customers at a time and then rolling it out widely after making sure that the update does not have unintended consequences. They can perform maintenance at a specific customer's convenience rather than at a time that works for all customers. More importantly, they can leverage the wide array of tools available to manage servers and VMs rather than rely on primitive tools available to manage networks.

Customized/New Services: The isolation of customers in VMs opens new options for service providers to provide customized services to each customer. For example, it is now possible for service providers to provide different versions of protocols for different customers without impacting each other. They can also turn on new services for customers (like DDoS detection, caching, redundancy elimination, etc.) relatively easily. The platform also opens up opportunities to support new technologies like information centric networking which requires the ability to both route traffic and store data within the network.

Simplified and Cost-Efficient Upgrades: EdgePlex supports existing network protocols. But the ability to support customized services for each customer allows service providers to run custom protocol versions for each customer. This allows providers to gradually transition their network to an SDN only environment since it decouples the speed of SDN adoption within their network and across individual customers. Customers wanting SDN can adopt it as soon as it is available, while others can continue using existing technology. It also eases transition for providers as they can gradually make features available.

4. IMPLEMENTATION AND EVALUATION

We built a prototype implementation of EdgePlex and performed experiments to understand the performance aspects of the architecture. We present details of our implementation and our experimental results in this section.

4.1 Prototype Implementation of EdgePlex

To demonstrate the efficacy of the EdgePlex design, we implemented a prototype of the EdgePlex platform. We used a combination of open source and custom software in our implementation. We used KVM as the hypervisor and ran multiple VMs on a single server. We used OpenBSD in the CVM. OpenBSD has in-built support for MPLS and for other routing protocols like OSPF and BGP. We configured OpenBSD appropriately to connect with route reflectors and other core network routers. Since, the CVM is primarily handling control-plane, we did not utilize any high-performance packet processing library in the CVM.

We used GNU/Linux running Quagga and a custom packet forwarding application (called FastForward) in our PVMs. Quagga handled individual customer BGP sessions and exchanges control plane between the customer and the CVM. FastForward was responsible for forwarding data packets from and to the customer. FastForward uses Intel's DPDK [1] for fast packet processing and Cuckoo hash library [26] for route lookup. When it receives packets from the customer, it has to insert MPLS label(s), update the ethernet headers and forward the packet to the backbone. FastForward gets MPLS label information from the CVM and stores it in the Cuckoo Hash. Based on the destination, FastForward uses the Cuckoo hash to find the right MPLS label(s), and inserts the labels into the packet. When it receives packets destined for the customer, it strips the MPLS labels, updates the ethernet headers, and forwards the packet.

4.2 Testbed Setup

We used the prototype implementation in a test bed consisting of x86 servers and OpenFlow capable switches to evaluate the performance of the platform. We interconnected the servers using OpenFlow-enabled switches to emulate a wide-area setup as shown in Figure 3. We used Ryu as our OpenFlow controller to talk to and configure the switches. We used the switches not only to interconnect the servers, but also to forward packets to the correct PVMs.

We partitioned the servers such that they host customer edge (CE) routers in two different cities. We then connected these CEs to a corresponding PVM in the EdgePlex platform in each of these cities. The two EdgePlex instances communicated with each other over the core network. We used a commercial, production-grade virtual router to function as our core router. This helps us demonstrate that EdgePlex can in fact work alongside existing infrastructure. We ran traffic end-to-end, including configured as an MPLS VPN with overlapping addresses, to demonstrate the viability of EdgePlex as a replacement for PE.

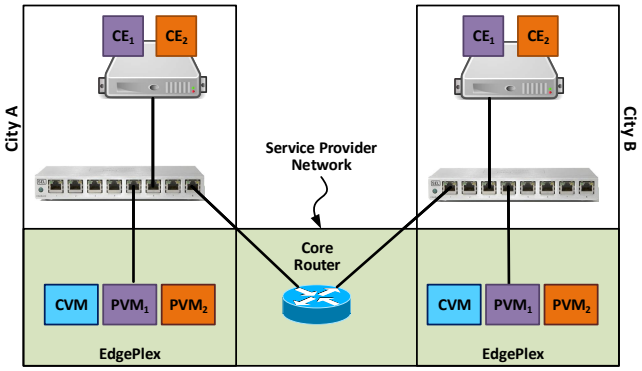


Figure 3: Testbed setup of a prototype implementation of EdgePlex.

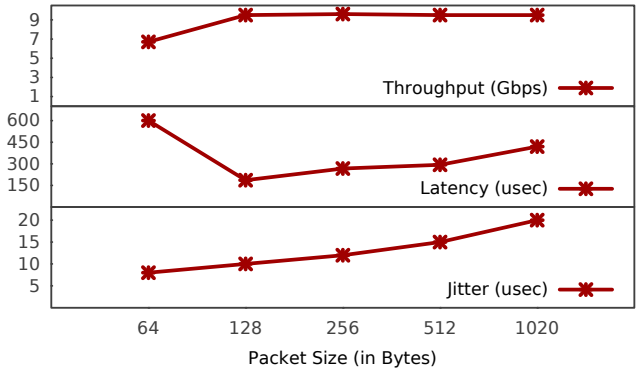


Figure 4: Throughput, Latency and Jitter for various packet sizes with 16 PVMs.

4.3 Data Plane Performance Evaluation

Since our servers had 16 cores, we varied the number of customers configured on EdgePlex between 1 and 16 to see the packet forwarding performance of EdgePlex. We used SR-IOV to virtualize the network interfaces across these PVMs. In the remainder of this section, we present results from performance experiments with 16 PVMs per server and measured at the ingress EdgePlex instance. The cuckoo hash in each PVM stores one million entries of target IP address to MPLS label mappings.

We use a modified version of pktgen (a DPDK-based packet generator) to generate traffic to the 16 PVMs. In addition to throughput, our modified version gives important information like latency of packet processing in a PVM and jitter between packets. To get these latter metrics, however, we had to bypass the end-to-end path and instead return the packets back to the generator.

Figure 4 shows the throughput achieved with various packet sizes with 16 PVMs. (Note that, like previous studies, we were able to achieve the full 10Gbps with 1 PVM). The computed value is the average of throughput achieved in 10 runs of the experiment. We see from Figure 4 that EdgePlex is able to achieve close to capacity of 10Gbps with packet sizes greater than 128-bytes. With 64-bytes packets, EdgePlex is only able to get up to 6.7 Gbps beyond which we start seeing packet drops. We attribute this to the overheads in processing the increased number of packets with 64-byte packets.

Throughput, however, tells us a part of the story. Hence we look into how long it takes to process the packets and any batching effects by looking into the latency and jitter experienced by packets and plot them also in Figure 4. We see that jitter is quite acceptable between 8 and 20 usecs for all packet sizes. Latency, however is more interesting. We see that there is a steady increase from about 180 usecs to 420 usecs as we go from 128 bytes to 1020 bytes. 64-byte packets however have higher latency at about 600 usecs. Despite these interesting data points, the main conclusion from our result is that EdgePlex platform is able to easily utilize the capacity of servers.

To understand the latency with 64-byte packets we studied latency and jitter under different input rates for 64-byte packets. Table 1 shows that the latency remains relatively stable until the input rate reaches the maximum throughput. At that point latency increases by more than ten fold. Interestingly we see that jitter drops as we increase the input traffic rate. This result leads us to believe that there is some form of queuing/batching in play. We plan to investigate the root cause further as part of our future work.

Input Rate(Gbps)	2G	3G	4G	5G	6G	7G[1]
Latency(us)	36	31	50	30	44	602
Jitter(us)	29	21	10	7	9	6

Table 1: Latency and jitter under different input rates with 64-byte packets.

4.4 PVM Migration and CVM Availability

A key feature of EdgePlex is that we can flexibly migrate PVMs from one host machine to another for tasks such as maintenance or load balancing. We evaluated the impact of migrating PVMs on the data forwarding performance using existing VM live migration capabilities. We noticed a connectivity interruption for less than one second when migrating a PVM. A major contribution of the connectivity interruption comes from the modification of switch forwarding rules. Recall that we used an OpenFlow capable switch to forward the packets to the correct PVM. We notify the switch to change its rules after the live migration is done. While this short down time is reasonable, we are investigating alternate mechanisms to avoid this interruption, including strategies proposed in OpenNF [13].

To guarantee the availability of EdgePlex, especially of the CVM, during failures, we used an existing tool in KVM called Micro-Checkpointing (similar to Remus [11] for Xen), which periodically generates a micro-checkpoint of CVM and copies it to a backup VM. We tested that with the help of Micro-Checkpointing, EdgePlex can recover from failures without affecting its running protocols. For example, the MPLS labels for different paths remain the same after the failure recovery. A key parameter of Micro-Checkpointing is the checkpoint frequency. On the one hand, if we generate checkpoints too frequently, the communication overhead to transfer dirty memory may be high. On the other hand, if we do not create enough checkpoints, we may lose some network state changes, such as routing updates. We measured the amount of transferred bytes for different checkpoint frequencies when CVM received 200 BGP updates per second. We tabulate our results in Table 2.

[1]We observed packet loss at this rate.

Checkpoint Interval (msec)	100	200	500	1000
Data Transferred (MB)	7.47	7.92	16.6	36.6

Table 2: Amount of data transferred with checkpointing the CVM for different time periods.

As expected the amount of data transferred increases as we increase the checkpointing interval. However, even in the worst case, the amount of data transferred is very manageable considering that this approach is used only for the CVM. In practice, we expect the routing protocol update frequency to be much lower and hence a lower communication overhead for Micro-Checkpointing.

5. CONCLUSIONS

We present EdgePlex, a novel approach for service provider edge router functionality. EdgePlex uses a distributed architecture composed of commodity servers and switches and takes advantage of advances in virtualization and high speed packet processing in servers and virtual machines to replicate the functionality in today's edge routers. We discuss the architecture of EdgePlex and highlight aspects of its design, in particular those that allow for scaling and efficient packet forwarding. A key novel aspect of EdgePlex is the use of a virtual machine per customer. This gives significant operational flexibility for providers and isolation for customers. Above all, our design allows for customization of services to individual customers and thereby provides service providers a path for gradual, yet cost-effective, migration from legacy technology to an all SDN solution. We build a prototype implementation of EdgePlex to demonstrate its efficacy and show through experiments that we are able to fully utilize server capacity (10Gbps) with minimal packet processing latency. Our results lead us to conclude that the EdgePlex design not only promises significant operational benefits over existing edge routers, it is also viable and capable of meeting the performance needs of service providers.

6. ACKNOWLEDGEMENTS

The authors would like to thank Aman Shaikh and Han Nguyen for valuable discussions and guidance on refining the EdgePlex design. We would also like to thank the anonymous reviewers for their feedback and suggestions on improving the paper.

7. REFERENCES

[1] DPDK. http://dpdk.org/.
[2] Open vSwitch. http://openvswitch.org/.
[3] PFQ. http://www.pfq.io/.
[4] PF_RING. http://www.ntop.org/products/pf_ring/.
[5] SR-IOV. https://www.pcisig.com/specifications/iov/.
[6] VMDq. http://www.intel.com/content/www/us/en/virtualization/vmdq-technology-paper.html.
[7] VMware. http://www.vmware.com/pdf/virtualization.pdf.
[8] M. Agrawal, S. R. Bailey, A. Greenberg, J. Pastor, P. Sebos, S. Seshan, J. van der Merwe, and J. Yates. RouterFarm: towards a dynamic, manageable network edge. In *Proceedings of the Workshop on Internet network management*, Pisa, Italy, 2006.
[9] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield. Xen and the Art of Virtualization. In *Proceedings of ACM Symposium on Operating Systems Principles*, 2003.
[10] Cisco. Cisco Cloud Services Router 1000V Series. http://www.cisco.com/c/en/us/products/routers/cloud-services-router-1000v-series/index.html.
[11] B. Cully, G. Lefebvre, D. Meyer, M. Feeley, N. Hutchinson, and A. Warfield. Remus: High availability via asynchronous virtual machine replication. In *Proceedings of USENIX Symposium on Networked Systems Design and Implementation*, April 2008.
[12] M. Dobrescu, N. Egi, K. Argyraki, B.-G. Chun, K. Fall, G. Iannaccone, A. Knies, M. Manesh, and S. Ratnasamy. RouteBricks: Exploiting Parallelism to Scale Software Routers. In *Proceedings of ACM Symposium on Operating Systems Principles*, 2009.
[13] A. Gember-Jacobson, R. Viswanathan, C. Prakash, R. Grandl, J. Khalid, S. Das, and A. Akella. OpenNF: Enabling Innovation in Network Function Control. In *Proceedings of ACM SIGCOMM*, Chicago, IL, USA, 2014.
[14] S. Han, K. Jang, K. Park, and S. Moon. PacketShader: A GPU-accelerated Software Router. In *Proceedings of the ACM SIGCOMM Conference*, 2010.
[15] J. Hwang, K. K. Ramakrishnan, and T. Wood. NetVM: High Performance and Flexible Networking Using Virtualization on Commodity Platforms. In *Proceedings of USENIX Symposium on Networked Systems Design and Implementation*, Seattle, WA, Apr. 2014.
[16] Juniper. Virtual Router for Enterprise & Service Provider Networks. http://www.juniper.net/us/en/products-services/routing/mx-series/vmx/.
[17] E. Keller, J. Rexford, and J. van der Merwe. Seamless BGP Migration With Router Grafting. In *Proceedings of USENIX Symposium on Networked Systems Design and Implementation*, San Jose, CA, USA, 2010.
[18] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori. kvm: the Linux Virtual Machine Monitor. In *Proceedings of the Linux Symposium*, volume 1, pages 225–230, Ottawa, Ontario, Canada, June 2007.
[19] E. Kohler, R. Morris, B. Chen, J. Jannotti, and M. F. Kaashoek. The Click Modular Router. *ACM Trans. Comput. Syst.*, 18(3):263–297, Aug. 2000.
[20] J. Martins, M. Ahmed, C. Raiciu, V. Olteanu, M. Honda, R. Bifulco, and F. Huici. ClickOS and the Art of Network Function Virtualization. In *Proceedings of USENIX Symposium on Networked Systems Design and Implementation*, 2014.
[21] K. K. Ram, A. L. Cox, M. Chadha, and S. Rixner. Hyper-switch: A Scalable Software Virtual Switching Architecture. In *Proceedings of USENIX Annual Technical Conference*, 2013.
[22] L. Rizzo. Netmap: A Novel Framework for Fast Packet I/O. In *Proceedings of USENIX Annual Technical Conference*, 2012.
[23] L. Rizzo and G. Lettieri. VALE, a Switched Ethernet for Virtual Machines. In *Proceedings of ACM Conference on Emerging Networking Experiments and Technologies*, 2012.
[24] P. Sebos, J. Yates, G. Li, M. Lazer, and D. Rubenstein. An Integrated IP/optical approach for efficient access router failure recovery. In *Optical Fiber Communication Conference*, 2004.
[25] Y. Wang, E. Keller, B. Biskeborn, J. van der Merwe, and J. Rexford. Virtual routers on the move: live router migration as a network-management primitive. In *Proceedings of ACM SIGCOMM*, Seattle, WA, USA, 2008.
[26] D. Zhou, B. Fan, H. Lim, M. Kaminsky, and D. G. Andersen. Scalable, High Performance Ethernet Forwarding with CuckooSwitch. In *Proceedings of ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '13, 2013.